

Chapitre 3

Diffusion

Notes rédigées par Raphaël JONTEF

cours dispensé par M. Cyril Gavaille

Enseirb-Matmeca
filière informatique

8 octobre 2026

Table des matières

I	Introduction	2
II	Arbre de diffusion	2
II.1	Message <i>vs.</i> connaissance	3

I Introduction

Supposons vouloir envoyer depuis un sommet distingué r_0 le même **petit** message M à tous les autres sommets d'un graphe $G = (V, E)$.

Proposition 3.1

Pour tout graphe G connexe et sommet distingué r_0 , tout algorithme distribué A de diffusion sur G et quelle que soit la connaissance à priori, on a :

- $\text{MESSAGE}(A, G) \geq n - 1$
- $\text{TEMPS}(A, G) \geq \text{ecc}_G(r_0)$

II Arbre de diffusion

On suppose qu'on dispose d'un arbre couvrant T enraciné en r_0 donné par $\begin{cases} u \mapsto \text{parent}(u) \\ u \mapsto \text{Enfants}(u) \end{cases}$ On considère l'algorithme de diffusion suivant :

```

1 | CAST(T, r0):
2 |   Si u ≠ r0:
3 |     M := RECEIVE()
4 |   Pour tout v dans Enfants(u):
5 |     SEND(M, v)
```

Proposition 3.2 - propriétés de cet algorithme

- $\text{TEMPS}(\text{CAST}(T, r_0), G) = \text{ecc}_T(r_0) = \text{hauteur}(T)$
- $\text{MESSAGE}(\text{CAST}(T, r_0), G) = |E(T)| = n - 1$

Cet algorithme a le mérite de limiter le nombre de messages à $n - 1$ mais il nécessite d'avoir T . Considérons à présent l'algorithme asynchrone suivant :

```

FLOOD(T, r0):
2 | si u = r0:
3 |   SENDALL(M) // envoi à tous les voisins
4 | sinon:
5 |   M := receive()
6 |   v := émettre du message M
7 |   SEND(M, w) pour tout w ∈ Voisins(u) \ {v}
```

Proposition 3.3 - propriétés de cet algorithme

On a, en notant m le nombre d'arêtes de G , et n son nombre de sommets :

- $\text{MESSAGE}(FLOOD(r_0), G) = m = \mathcal{O}(n^2)$
- $\text{TEMPS}(FLOOD(r_0), G) = \text{ecc}_G(r_0)$

Démonstration

1.

$$\begin{aligned} \text{MESSAGE}(FLOOD(r_0), G) &= \deg_G(r_0) \sum_{\substack{u \in V \\ u \neq r_0}} (\deg_G(u) - 1) \\ &= \sum_{u \in V} \deg(u) - (n - 1) \\ &= 2m - n + 1 \end{aligned}$$

2. Montre qu'après t unités de temps, tous les sommets à distances $\leq t$ ont reçu M . Il y a donc au plus $\text{ecc}_G(r_0)$ unités de temps avant que tous les sommets aient reçu M .

Il existe également une variante synchrone "amnésique" de FLOOD qui consiste à appliquer FLOOD perpétuellement.

```

1 AMNESIAC-FLOOD(r0):
2 si u = r0:
3   SENDALL(M)
4 S = ∅
5 tant que vrai:
6   démarrer une nouvelle ronde
7   si S = ∅:
8     SEND(M, v) pour tout v ∈ Voisins(u) \ S
9     // c'est grâce à ça qu'on s'arrête
10  M := RECEIVEALL() // on reçoit tous les messages des voisins
11  S := l'ensemble des émetteurs de M
12  si S ≠ ∅:
13    choisir M ∈ M

```

II.1 Message vs. connaissance

	TEMPS	MESSAGE	"CONNAISSANCE"
CAST _T	$\text{ecc}_T(r_0)$	$n - 1 = \mathcal{O}(n)$	T
FLOOD	$\text{ecc}_G(r_0)$	$2m - n + 1 = \mathcal{O}(n^2)$	$\emptyset$

Définition 3.4 - *connaissance à distance ρ*

On dit qu'un sommet u a connaissance à distance ρ si u connaît l'ensemble des sommets à distance $\leq \rho$ de lui, ainsi que les arêtes entre ces sommets.

Proposition 3.5 - *petits messages*

$\forall \rho \in \mathbb{N}, \forall G$ connexe à n sommets et r_0 , il existe un algorithme de diffusion FLOODK_ρ avec une connaissance ρ tel que :

$$\text{MESSAGE}(\text{FLOODK}_\rho(r_0), G) = 2 \min(m, n^{1+\frac{1}{\rho}})$$

Ainsi :

- FLOODK_0 est en $\mathcal{O}(n^2)$
- FLOODK_1 est l'algorithme FLOOD, également en $\mathcal{O}(n^2)$ qui ne nécessite aucune connaissance.
- FLOODK_2 est en $\mathcal{O}(n^{3/2})$
- FLOODK_3 est en $\mathcal{O}(n^{4/3})$
- FLOODK_∞ est l'algorithme CAST_T , en $\mathcal{O}(n)$

Démonstration

On prend $\rho \geq 1$. Deux idées :

- Localement, chaque sommet décide de désactiver certaines arêtes, et $\tilde{G}$ le graphe résultant.
- On applique FLOOD dans $\tilde{G}$, $\text{MSG} \leq 2\tilde{m} - n + 1$